Chapter III. Source and target languages

In this chapter we introduce source and target languages *LS* and *LT*. We first give an informal overview over the source language followed by its formal definition. The target language is described in a similar fashion.

1. The source language

One objective of this work is to write a compiler for a realistic and useful language. Since Pascal is a fairly simple but still useful language our source language *LS* is based on Pascal.

A second criterium is that we need a formal definition of *LS*. There exist formal definitions of Pascal of various flavours [HW73, AA77, Te77a]. We build on these, notably Tennent's denotational semantics [Te77a].

However, we made some changes to Pascal. Several language features of Pascal have been omitted. This has quantitative rather than qualitative reasons and was done to keep the task of verifying the compiler manageable. The theory and proof techniques developed in this thesis is general enough to handle all omitted cases.

Some parts of Pascal are poorly defined (see [Ha73, WS77]) and have been changed or extended[1]. Some of the extensions follow naturally from an attempt to formally define the language as has been pointed out in [Te77b].

This section gives an informal overview of *LS* and points out differences to Pascal.

1.1. Data structures

As Habermann [Ha73] points out, the notion of type is not very well defined in Pascal. Aside from his largely informal arguments this also becomes apparent if one tries to give a formal definition of the type concept of Pascal. In fact, in Tennent's definition the Pascal concept is completely abandoned and a structural type equality in the sense of Algol 68 is substituted.

1.) Most changes are similar to the ISO standard for Pascal [Ra79, Ad80].

Types in Algol 68 are equal if they are of the same structure. That is two records are equal if they have the same field selectors and equal selctors refer to components of equal type. Two pointer types are equal if they point to equal types.

This concept seems appealing, however, it poses some problems. It is non-trivial to determine the equality of two types. Engelfried [En72] has shown that this is equivalent to determine equivalent states of a finite automaton.[2]

Defining "structural" equality formally is not at all straightforward. In the original report [Wi69] type equality was "defined" in English while the revised report [Wi76] uses a non-intuitive definition in terms of vanWijngaarden predicates. In Tennent's denotational definition of Pascal [Te77a] the Algol 68 style equality is used. One problem in his semantics is that the definition of a monotonic equality on types requires to count the number of types defined in a program which seems rather unnatural. Consider the two types **type** $p = \uparrow q$; $q = \uparrow p$;. Two types are equal in the Algol 68 sense if they are both pointers to equal types, thus we have $equal(p, q) \equiv equal(q, p)$ which is not well defined. Tennent uses the number n of types defined in an environment and defines $equal(p, q, n) =$ **if** $n =$ 0 **then** *true* **else** $equal(q, p, n - 1)$.

Pascal's types are very simple to compare; each type declaration defines a new unique type, not equal to any other type. Besides being easy to implement it allows the programmer to take advantage of additional type checking possibilities: one may have two types **array** [1..5] **of** *integer* which are different. The Pascal view of types causes several problems if we consider subranges. A constant of an enumerated type may also be of a subrange type. Operations on enumerated types must be viewed as generic (for all subranges) or elements of a subrange have to be coerced to the supertype before such operations can be applied. Haberman [Ha73] presents various examples involving these problems.

Our solution to types is as follows. Two type declarations define different types as in Pascal. However, subranges do not constitute new types; rather, a subrange type is equal to the supertype of which it is a subrange. Each scalar type in *LS* has a "subrange attribute" which specified the allowable range of values of this type. Variables of type subrange are considered variables of the supertype except that assignments to them causes a runtime check for the bounds.

LS has the predefined types *Integer* and *Boolean*. It allows the definition of enumerated types and subranges [JW76]. Furthermore the user can define arrays with constant bounds, records without variants, and pointer types. Even though variable bounds for arrays would be a reasonalbe extension and make the implementation slightly more interesting they have been omitted. Adding variable

2.) Actually the problem is further complicated by union types which are left out of Engelfried's considerations.

bounds means that subranges can no longer be used to define the index range and a new semantic concept has to be introduced in the language.

Notable omissions from Pascal are characters, reals and file types. Also, we do not consider records with variants as their definition is Pascal is unsafe (i.e. violates strong typing).

LS allows the definition of constants of type boolean and integer. This concept is a slight generalization of Pascal in that an arbitrary expression is allowed to define a constant provided this expression can be evaluated at compile time.

A major deficiency of Pascal is that there are no constant denotations for arrays and records. In fact, the type concept of Pascal makes it hard to incorporate constant arrays. For example what should the type be of say $(1, 2, 3, 4, 5)$. Certainly it is not **array** $[1..5]$ **of** *integer*; one possible solution would require constructors for constants explicitly naming the type. Constant denotations for complex types are omitted in *LS*.

1.2. Program structures

Statements of *LS* are those of Pascal with the exception of *for*-loops, *case* and *with*-statements. These are omitted since they are not strictly necessary in the language and their implementation poses no particular problem (possible extensions of our language to include the above features are discussed in chapter V). The *while* and *if* statement are changed to contain a syntactical end symbol which allows *while* bodies and *if* branches to be statement lists without having to write *begin* and *end*.

Pascal's block structure has been extended to allow declarations in inner blocks. *LS* allows jumps as in Pascal; jumps into blocks or statements are illegal. Labels do not have to be declared and may be reused in inner scopes.

Procedures and functions are recursive but not mutually recursive. The reason is that we omit forward declarations of procedures. Alternatively, we could allow procedure and function identifiers to be used before their declaration. Either extension would merely complicate the static semantics; our code generation can handle mutual calls.

Procedures and functions may not be passed as parameters. One problem with procedure parameters is that this mechanism in its full generality requires type checking at runtime. Alternatively one could require the specification of the parameter types of the formal procedure parameters as in Algol68. For example

procedure *p*(**var** *x* : *integer*; *f* : **function**(*integer*, *boolean*) : *integer*);

Each *LS* program has one anonymous input and one anonymous output file both of type integer. *LS* provides the operations *read*,*eof*, and *write* operating on these files. Opening and closing of files is done automatically at the beginning and

end of program execution. Definition, implementation and proof can be extended easily for other predefined functions and procedures; more I/O operations can be added. An extension of the input output facilities to those of Pascal requires the introduction of file types as well as the formal definition of certain component of the operating system environment. For example the semantics of opening a nonexisting file has to be defined.

Expressions in *LS* are those of Pascal with exception of the precedence of operators which is more natural in *LS*.

1.3. Structure of the formal definition

The definition of *LS* is divided into the definition of syntax and semantics. The syntax is further subdivided into micro syntax, concrete syntax, and tree transformation. Semantics is subdivided into static and dynamic semantics. Each of these components defines a mapping; the composition of these mappings defines the meaning of a program. We now define the individual mappings and their composition.

The following domains are common to all components of the compiler. Internally, the definitions of individual components utilize additional domains.

$c \in Ch$	Characters
$s \in Str = Ch^*$	Strings
Tm	Terminal symols
Vl	Token values
$Tk = Tm \times Vl$	Tokens
St	Syntax trees
Pgm	Abstract programs
E_T	Token errors
E_S	Syntax errors
E_C	Compile time errors
E_R	Runtime errors
$E = E_T + E_S + E_C + E_R$	Errors
$M = N^* \to (N^* + E_R)$	Meaning of programs

Most of these domains are explained in more detail in the corresponding section. For the present discussion their internal structure is irrelevant. It was necessary on this level to define M, the meaning of programs, since this shows that errors can not only occur during compilation but also during execution. The meaning of a program then is a mapping from the input file, N^* into the output file N^* or a possible error.

The language is completely specified by defining the above domains and the following functions:

$scan \in Str \to (Tk^* + E_T)$	Scanner
$parse \in Tk^* \to (St + E_S)$	Parser
$treetr \in St \to Pgm$	Tree transformations
$ssem \in Pgm \to Pgm + E_C$	Static semantics
$dsem \in Pgm \to M + E_R$	Dynamic semantics

[E–strict function composition] Let $f \in D_2 \to D_3$ and $g \in D_1 \to D_2 + E$ then *E–strict function composition*, $f \odot g \in D_1 \to D_3 + E$ is defined as

$$f \odot g = \lambda x.\ \textbf{let}\ t = g\ x\ \textbf{in}\ \ \textbf{if}\ t \in E\ \textbf{then}\ t\ \textbf{else}\ f\ t.$$

The meaning of a program is a mapping from character strings into M given as:

$$\boxed{smean \in Str \to M + E}$$

$$smean = dsem \odot ssem \odot treetr \odot parse \odot scan$$

In the following sections we present the formal definition of the inidividual components. In each case we introduce the definitional formalisms used and explain key points in the definition; a complete formal definition of LS can be found in the appendix.

2. Micro syntax

The micro syntax defines a mapping from a sequence of characters into a sequence of tokens. A token is a pair containing syntactic and semantic information.

2.1. Definitional formalism

Micro syntax is defined in two steps. First we define which substrings of the input constitute single tokens. Each of these strings is then mapped into the corresponding syntax semantic pair.

We consider a finite number of classes of tokens; the external representation of elements of each class is given as a regular language L_i. For example an identifier is a string of digits and letters, starting with a letter. Similarly, numbers and single and multi-character delimiters are defined.

Let $\{L_i\}$ be a set of regular languages each defining a class of tokens. We define rules according to which the input string is to be broken into substrings each of

which is in one language L_i. The rule is to find the longest leftmost substring s of the input such that $s \in L_i$ for some i. Given $s \in L_i$, a *semantic* function S_i maps the string s into the token represented by s.

For example, L_{id} is the language of identifiers. S_{id} checks if an identifier is a reserved word and treats it differently from ordinary identifiers. Further, an identifier which is not reserved is encoded as a unique integer by S_{id}. Clearly, to do this the functionality $L_{id} \to Tk$ is too simple for S_{id}. Rather S_{id} needs additional infomation about identifiers that have been scanned previously as well as which codes have already been used to encode identifiers.

We introduce identifier tables

$$h \in H = (L_{id} \to N) \times (N \to \{used, unused\})$$

mapping identifiers into N and specify whether a number in N is *used* or *unused*. Now, the functionality of S_i becomes

$$S_i \in H \to L_i \to (Tk \times H).$$

Let us now describe how a given set of languages L_i and semantics functions S_i are combined to define the scanning function $scan \in Str \to (Tk^* + E_T)$. We define

$$scan = \Psi\ h_0$$

where h_0 is the initial identifier table which contains all predefined identifiers of the language.

$\Psi \in H \to Str \to Tk^*$ deletes initial substrings from the input string until the first character c is the beginning of a token. c is the beginning of a token if $c \in \bigcup \alpha_i$, where α_i is the set of all non–empty initial segments of elements of L_i:

$$\alpha_i = \{u \mid \exists v.u{\cdot}v \in L_i, u \neq \langle\rangle\}$$

Given $c \in \bigcup \alpha_i$ function Φ is applied to recognize the token beginning with c.

$$\begin{aligned}\Psi\ h\ s = {} & \textbf{if } s = \langle\rangle \textbf{ then } \langle\rangle \textbf{ else} \\ & \quad \textbf{if } hd\ s \in \bigcup \alpha_i \textbf{ then } (\Phi\ h\ (hd\ s)\ (tl\ s))^{\#1} \textbf{ else} \\ & \quad \Psi\ h\ (tl\ s)\end{aligned}$$

$\Phi \in H \to Str \to Str \to (Tk^* \times H)$ applied to two strings s_1 and s_2 finds the longest initial substring of $s_1{\cdot}s_2$ which is in some α_i. Φ repeatedly appends the first character of s_2 to s_1 until $s_1 \in \bigcup \alpha_i$ (s_1 is the beginning of a token) and $s_1{\cdot}\langle hhd\ s_2\rangle \notin \bigcup \alpha_i$ (i.e. s_1 is the longest possible substring). In this situation Φ returns the token corresponding to s_1 concatenated with the tokens given by the remainder (s_2).

$$\Phi\ h\ s_1\ s_2 = \textbf{if } s_2 = \langle\rangle \textbf{ then } \textbf{if } s_1 \in L_i \textbf{ then } (S_i\ h\ s_1)^{\#1} \textbf{ else } \textit{error} \textbf{ else}$$
$$\textbf{if } s_1{\cdot}(hd\ s_2) \in \bigcup \alpha_i \textbf{ then } \Phi\ h\ (s_1{\cdot}(hd\ s_2))\ (tl\ s_2) \textbf{ else}$$
$$\textbf{if } s_1 \in L_i \textbf{ then } (S_i\ h\ s_1)^{\#1}{\cdot}(\Psi\ (S_i\ h\ s_1)^{\#2}\ s2) \textbf{ else } \textit{error}$$

2.2. Micro syntax of *LS*

To define the micro syntax we have to specify the domains Tm, Vl, languages $\{L_i\}$, semantic functions $\{S_i\}$, and the initial identifier table h_0.

For *LS* we have 5 regular languages.

L_{id} is the set of all identifiers. The corresponding semantic function S_{id} will detect reserved words and encode all other identifiers as unique integers, using the identifier table h.

L_n is the set of numbers. S_n defines which integer value is denoted by a given string of digits.

L_d is the set of delimiters consisting of just one character. S_d will map each character into its representation as a token.

L_p is the set of all tokens beginning with a period, i.e. {".", ".."}.

L_c is the set of all tokens beginning with a colon, i.e. {":", ":="}.

The complete formal definition of the scanner for *LS* can be found in appendix 1.

3. Syntax

The syntax of *LS* is defined by a context free grammar. We extend the definition of grammars to include unique labels for each production. We require that the grammar be unambiguous. In this case the grammar defines a mapping from token sequences Tk^* to parse trees. The production labels become labels of parse trees.

3.1. Labeled context free grammars

Ordinarily a grammar is given by a set of productions. Instead, we specify labeled grammars by a set of lables L together with a mapping P from labels to productions.

$n \in Nt$ Nonterminal symbols

Tm	Terminal symbols
$\lambda \in L$	Labels
$u, v, w \in W = (Nt \bigcup Tm)^*$	Words
$P \in L \mapsto Nt \times W$	Productions

$s_0 \in Nt$ is a distinguished *start symbol*. If $P\,\lambda = \langle n, w\rangle$ we write

$$[\lambda]\ n ::= w.$$

A grammar G is completely specified by the elements Nt, Tm, L, P, and s_0.

3.1.1. The accepted language

On W we define a binary (infix) relation $\rightarrow \in W \times W$ as follows.

$$u \cdot \langle n \rangle \cdot v \rightarrow u \cdot w \cdot v \text{ iff } \exists \lambda \in L . P(\lambda) = \langle n, w \rangle.$$

Let $\rightarrow^*$ be the reflexive, transitive closure of $\rightarrow$. For a grammar G we define

$$L(G) = \{w \in Tm^* \mid s_0 \rightarrow^* w\}.$$

Note, that $L(G)$ is a language over strings of terminal symbols.

Recall that the domain of tokens was defined as $Tk = Tm \times Vl$. We extend $L(G)$ for strings of tokens and define:

The language accepted by a labeled context free grammar $L_{Vl}(G)$ is

$$L_{Vl}(G) = L(G) \times^* Vl^*$$

Here $\times^*$ is the obvious componentwise extension of the cartesian product to sequences:

$$S \times^* T = \{\langle s_1, t_1\rangle \ldots \langle s_n, t_n\rangle \mid s_1 \ldots s_n \in S, t_1 \ldots t_n \in T\}.$$

3.1.2. Parse trees

A tree $s \in S = W_L(Tk)$ is a partial parse tree if

- whenever $\acute{s} = (\lambda\ \tau_1\ \ldots\ \tau_m)$ is a subtree of s, then $P(\lambda) = \langle n, x_1 \cdot \ldots \cdot x_m\rangle$ and for all $0 \leq i \leq m$ we have either $\tau_i \in Tk$ and ${\tau_i}^{\#1} = x_i$ or $root\ \tau_i = x_i$.

A partial parse tree s is a parse tree if in addition

- $root\ s = \langle s_0, w\rangle$ for some w.

3.1.3. The function defined by a labeled grammar

We require that a labeled context free grammar be unambiguous, that is, whenever $s_1, s_2 \in W_L(Tk)$ then $leaves\ s_1 = leaves\ s_2$ implies $s_1 = s_2$.

Each unambiguous labeled context free grammar G defines a "parsing function" $parse \in Tk^* \to St$ as follows.

$$parse(w) = \textbf{if } w \in L(G) \textbf{ then } s, \text{ such that } leaves(s) = w \textbf{ else } error$$

where s is a parse tree for G.

3.2. Syntax of *LS*

Using the above theory, the syntax of *LS* is defined by giving a labeled context free grammar for *LS*. The function *parse* for *LS* is the function defined by this grammar. A labeled context free grammar for *LS* is given in appendix 1. This grammar also satisfies the SLR(1) condition [De71]; we will use this property in the construction of a parser for *LS* which is described in chapter IV.

4. Tree transformations

Tree transformations define a mapping from parse trees into *abstract syntax*. We first define the abstract syntax of *LS* and then specify tree transformations for *LS*.

4.1. Abstract syntax

The idea of an *abstract syntax* has first been introduced by McCarthy in [Mc66]. It allows to reason about programs regardless of their external representation (on paper or in a machine). Formally, abstract syntax can be defined as a free algebra; for our purposes abstract syntax is a domain *asyn* of programs.

asyn is the sum of a set of syntactic domains. These domains define syntactic entities of *LS*, such as identifiers, expressions, statements and so on. We use a slightly different notation here than we use for semantic domains. For example instead of writing

$$\Theta \in Stm = (Exp \times Exp) + (Exp \times Com) + \ldots$$

we write

$$\Theta ::= E_0 := E_1 \mid while\ E\ do\Gamma\ od \mid \ldots$$

for $E_i \in Exp$ and $\Gamma \in Com$. Thus, this notation not only defines the domains but also a particular representation which is used to refer to elements of these domains.

4.1.1. A different notation

In order to mechanize reasoning about abstract syntax it is convenient to introduce a different notation for elements of *asyn*. For example, consider *while* E *do* Γ *od*. We imagine that this element is constructed from E and Γ by a function *mkewhile*.

More generally, we provide a name for each syntactic construct. Let *xxx* be such a name for a construct with n components, we agree to use the symbol *mkexxx* as a n-ary function on *asyn* "constructing" the appropriate element.

Furthermore we agree that *decxxx* is the inverse operation to *mkexxx*; it returns the list of components of a given element of *asyn*:

$$decxxx(mkexxx(\tau_1, \ldots, \tau_n)) = \tau_1 \cdot \ldots \cdot \tau_n.$$

Finally, we use the convention that *isxxx* is a predicate which is true for all syntactic elements of kind *xxx* and false otherwise.

In later sections we intermix the constructor notation with the above informal one. For example, we use "*while* E *do* Γ *od*" in the formal language definition while we use "*mkewhile*(E, Γ)" in the machine readable lemmas input to the verifier.

The *conformity relation* :: is used to conveniently access components of a syntactic element. For $E \in asyn$ we define

$$E :: mkexxx(x_1, \ldots, x_n, y_1, \ldots, y_m)$$

to be true in a context where all y_i are bound and x_i are free if and only if $\exists x_1 \ldots x_n . E = mkexxx(x_1, \ldots, x_n, y_1, \ldots, y_m)$. In addition, conformity relations cause free variables to be bound to corresponding components. For example,

$$\textbf{if } E_0 :: mkexxx(x_1, \ldots, x_n, y_1, \ldots, y_m) \textbf{ then } E_1 \textbf{ else } E_2$$

means

$$\begin{aligned} \textbf{if } \exists x_1 \ldots x_n . E_0 = mkexxx(x_1, \ldots, x_n, y_1, \ldots, y_m) \textbf{ then} \\ \quad \textbf{let } x_i = decxxx(E)^{\#i} \textbf{ in } E_1 \\ \quad \textbf{else } E_2 \end{aligned}$$

where again x_i are free variables.

4.1.2. Syntactic domains of *LS*

$Pgm = Stm$	Programs
$\Omega \in Op$	dyadic operators (not further defined here)
$O \in Mop$	monadic operators (not further defined here)
$I \in Id$	Identifiers (not further defined here)
$N \in Num$	Numerals (not further defined here)

$\Xi \in Pgm$	Programs
$E \in Exp$	Expressions
$\Theta \in Stm$	Statements
$\Gamma \in Com$	Commands
$T \in Typ$	Types
$\Delta \in Dec$	Declarations
$\Delta_c \in Cdef$	Constant Definition
$\Delta_t \in Tdef$	Type definition
$\Delta_v \in Vdec$	Variable declaration
$\Pi \in Par$	Parameters

Expressions

$$E ::= I \mid O\ E \mid E_0\ \Omega\ E_1 \mid E \uparrow \mid I(E^*) \mid E_0[E_1] \mid N \mid E.I$$

Commands

$$\Gamma ::= N : \Theta \mid \Theta \mid \Gamma_0;\Gamma_1$$

Statements

$$\Theta ::= E_0 := E_1 \mid \mathit{if}\ E\ \mathit{then}\ \Gamma_0\ \mathit{else}\ \Gamma_1\ \mathit{fi} \mid \mathit{dummy} \mid \mathit{while}\ E\ \mathit{do}\ \Gamma\ \mathit{od} \mid \mathit{repeat}\ \Gamma\ \mathit{until}\ E \mid \mathit{goto}\ N \mid I(E^*) \mid \Delta^*\ \mathit{begin}\ \Gamma\ \mathit{end}$$

Declarations

$$\Delta ::= \mathit{const}\ \Delta_c^* \mid \mathit{type}\ \Delta_t^* \mid \mathit{var}\ \Delta_v^* \mid \mathit{procedure}\ I(\Pi^*);\Theta \mid \mathit{function}\ I(\Pi^*) : T;\Theta$$

Types

$$T ::= I \mid (I_1, \ldots, I_n) \mid E_1 . . E_2 \mid \mathit{array}[T_1]\mathit{of}\ T_2 \mid \mathit{record}\ I_1 : T_1; I_2 : T_2; \ldots; I_n : T_n\ \mathit{end} \mid \uparrow I$$

Constant Definitions

$$\Delta_c ::= I = E$$

Type Definitions

$$\Delta_t ::= I = T$$

Variable Declarations

$$\Delta_v ::= I : T$$

Parameters

$$\Pi ::= I_1 : I_2 \mid \mathit{var}\ I_1 : I_2$$

In addition to this mathematical notation a machine readable representation

is necessary for mechanical proofs. This is provided by constructor and selector functions which are defined in appendix 1.

The domains *Id* and *Num* are left undefined. The structure of identifiers is part of the definition of the micro syntax. All that is relevant for the semantics of *LS* is that we can test identifiers for equality. Similarly, the structure of numerals is irrelevant. We are only interested in the value denoted by a numeral.

4.2. Tree transformations for *LS*

A tree transformations is defined mapping parse trees into elements of the abstract syntax. The function Ξ is defined recursively on parse trees. One definitional clause is provided for possible node (label of the grammar). For example, the grammar of *LS* contains the following productions:

[*STMT_5*] STMT::= ifsymbol EXPR thensymbol COM fisymbol
[*STMT_6*] STMT::= ifsymbol EXPR thensymbol COM
elsesymbol COM fisymbol
[*FACT_1*] FACT::= lparensymbol EXPR rparensymbol

The corresponding clauses of the tree transformation are

$$\Xi(STMT_5, \tau_1, \tau_2, \tau_3, \tau_4, \tau_5) = mkeif(\Xi(\tau_2), \Xi(\tau_4), mkestmt(mkedummy))$$
$$\Xi(STMT_6, \tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7) = mkeif(\Xi(\tau_2), \Xi(\tau_4), \Xi(\tau_6))$$
$$\Xi(FACT_1, \tau_1, \tau_2, \tau_3) = \Xi(\tau_2).$$

For example, in the abstract syntax there is no **if** statement without **else** part. The first clause above maps a conditional without matching **else** part into an **if** statement with empty **else** part. The third line shows that parenthesis in expressions are ignored in the abstract syntax. The complete definition of Ξ can be found in appendix 1.

5. Semantics of *LS*

The semantic definition of *LS* follows [Te77a]. In particular we use Tennents separation of dynamic and static semantics. The intuition is, that a function is static if it can be evaluated at compile time, dynamic otherwise; but formally this division is arbitrary, since there is no notions of compile and run time in a formal semantics.

The dynamic semantics of *LS* is specified on a fairly low level using concepts well known to the compiler constructor. Clearly, a more "abstract" definition of *LS* is possible. In this case the equivalence with a lower level description such as

ours has to be proven as part of the compiler verification. Proof techniques to do this are well established (see for example [MS76]).

Since we are more interested in techniques of specification and verification rather than semantics theories we choose to base the compiler proof on a more detailed definition. This definition can be used directly as the formal basis for the compiler.

The formal definition of *LS* is presented in appendix 1. In this section we explain some of the interesting points of this definition. We will be informal in our explanations and relate formal objects of the definition to concepts known to compiler constructors.

5.1. Semantic concepts

5.1.1. Semantic domains

In this section we describe the semantic domains used in the definition of the semantics of *LS*.

T and N are truth values and integers respectively. T is used to model predicates of the descriptive language; the data type *boolean* is not related to T.

$I = \{int_{-\infty}, \ldots, int_{+\infty}\} \subseteq N$ is the set of index values. These are those integers that can be represented in our target machine. The specific values of $int_{+\infty}$ and $int_{-\infty}$ are irrelevant; we assume however, that $0, 1 \in I$.

$Tg = \{int, bool, \nu_1, \ldots, \nu_i, \ldots\}$ is a flat domain of type tags. Type tags are used to create new names for types in *LS*; i.e. as in Pascal each new type definition creates a unique type. Note, that we cannot simply use the identifier provided in the *LS* program to name types since there are types without explicit name and scope rules would cause ambiguities.

The concept of locations in *LS* is fairly complex. First we have a set of absolute locations. These can be thought of as addresses of a real physical memory. Abstract locations are divided into two distinct sets L_d and L_s, called dynamic and static locations respectively. The difference between L_d and L_s is that dynamic objects (created with the *new* statement) are assigned dynamic locations (on the heap). Variables declared in a block or procedure are allocated static locations (on the stack).

A location in L_d or L_s is able to hold a single object (such as an integer or boolean value). To be able to describe addresses assigned to complex objects such as record and arrays a more complex structure of locations is required. We call those more general locations L-values (Lv). Again, we distinguish static and dynamic

L–values, Lv_s and Lv_d. We define

$$v \in Lv_s = L_s + (Id \to Lv_s) + (I \to Lv_s)$$
$$v \in Lv_d = L_d + (Id \to Lv_d) + (I \to Lv_d) + \{NIL\}$$

This means that a static L–value is either a single location (L_s) or the address of a record ($Id \to Lv_s$) or the adddress of an array ($I \to Lv_s$). The address of a record is a mapping which given a component identifier returns the address of this component. Similarly, the address of an array is a mapping from index values into addresses of the components of the array.

In addition to static and dynamic locations we introduce *relative locations* L_r and relative L–values Lv_r. Since procedures and functions in LS can be recursive different incarnations of the same procedure (or function) give raise to different absolute locations for the local variables of this procedure. To describe the meaning of a procedure statically, i.e. independently of a particular call, we assign relative locations and L–values to local variables. At run time these relative locations are mapped into absolute locations by a *memory frame*. We define memory frames as

$$f \in F = L_r \to L_s.$$

Note, that a realtive location is always mapped into a static location since there are no variables that are assigned dynamic locations.

Although memory frames are defined for L_r only they immediately extend to relative L–values as follows. ($\hat{f}$ is the extended function)

$$\begin{aligned}\hat{f}\alpha = \ & \textbf{if } \alpha \in L_r \textbf{ then } f\alpha \textbf{ else} \\ & \textbf{if } \alpha \in (I \to Lv_r) \textbf{ then } \lambda\iota.\hat{f}(\alpha\iota) \textbf{ else} \\ & \textbf{if } \alpha \in (Id \to Lv_r) \textbf{ then } \lambda I.\hat{f}(\alpha[\![I]\!])\,.\end{aligned}$$

We assume that there are infinitely many frames $f_i \in F$ and that f_i are totally ordered. The function *succ* gives the successor of a frame, i.e. $succ\ f_i = f_{i+1}$. Further, the ranges of two different frames are distinct, i.e. for all f_i, f_j and α_1 and α_2 we have that $f_i \neq f_j \Rightarrow f_i\alpha_1 \neq f_j\alpha_2$.

Values V are objects that can be stored in memory and can be result of an expression. The only values in LS are index values and pointers, i.e. we have $V = I + Lv$. In particular there are no array or record values; consequently, arrays and record as whole structures cannot be assigned or passed as value parameters.

A store can be thought of as the memory of an abstract machine; it is a mapping from locations to values. To characterize a computation state completely, a store in LS has two components representing the current input and output file. We define

$$S = (L_s + L_d) \to (V + \{unused\}) \times N^* \times N^*$$

A dynamic location which is unused is mapped into *unused*. The use of static locations will be recorded in the environment (see below).

An answer of a *LS* program can either be an output file $\in N^*$ or a runtime error, thus we have $A = N^* + E_R$.

Continuations $C = S \to A$ will be referred to as dynamic continuations. Instead of introducing expression continuations we define dynamic generalized continuations as:

$$G_d = V^* \to C$$

It is easy to see that G_d is isomophic to

$$V \to \ldots \to V \to C$$

for any number of V's. The use of dynamic generalized continuations will greatly simplify the definition of meaning functions. In addition generalized continuations have an obvious realization in a compiler. Note, that $V^* \to C$ can be interpreted as an expression continuation which takes a "stack" ($\in V^*$) as argument. As can be seen later when we give the definition of meaning functions this stack is exactly the stack used to evaluate expressions in a compiler.

Each call to a procedure requires that new memory be allocated; an executing procedure together with its local memory is called an incarnation of this procedure. Addresses used in the code of a procedure are relative addresses ($\in L_r$). To access the content of a relative locations it has to be converted to an absolute locations using a frame.

The incarnation of the procedure that called the currently executing procedure p is said to be the *dynamic predecessor* of the current incarnation of p.

Every procedure has an associated lexical level; the main program has lexical level 0. If a procedure p with lexical level n is called, then at least one incarnation of all procedures with smaller lexical level ($< n$) which contain the declaration of p has to exist[3]. If q is such a procedure with lexical level m, $m < n$, then the most recent incarnation of q is said to be the static predecessor of p of level m.

If a variable with relative location α is defined on lexical level m and is accessed in a procedure p with lexical level n, $m < n$, then the frame belonging to the static predecesor of p of level m has to be used to covert α into an absolute location. We introduce a domain of displays to describe this use of frames. The concept of a display is well known to the compiler constructor, see for example [RR64,Gr71].

We define

$$\chi \in X = (B \to X) \times (B \to F) \times X \times B.$$

3.) This is not true for languages that allow for procedural parameters.

Here $B = N$ is the domain of lexical levels. A display χ has the following components

- A mapping from lexical levels to displays which gives the static predecessor of the current display.
- A mapping from lexical levels to frames giving the frame corresponding to the static predecessors.
- The dynamic predecessor.
- The lexical level of the currently executed procedure.

At any point the meaning of the remainder of the program depends on the current display. Therefore, we introduce generalized continuations G which depend on a particular display:

$$G = X \to G_d.$$

The meaning of a procedure is a mapping from a list of parameter values and a continuation into the domain of continuations. We could define $V^* \to G \to G$. But this domain is isomorphic to $G \to V^* \to G$ which in turn is isomorphic to $G \to G$. Thus we define $P = G \to G$ as the domain of procedure values. Functions can be modeled by the very same domain. We do not distinguish procedure and function values.

We use two kinds of environments in the definition of LS. Static environments U_s contain information about identifiers and numbers relevant for static semantic analysis. Environments U are used to describe dynamic semantics. In

$$U_s = (Id \to Md) \times (Num \to T) \times (Tg \to T)$$

each identifier is mapped into a mode; each number is mapped to a truthvalue indicating whether or not this number is a defined label. Type tags Tg are mapped into truthvalues to indicate which of the tags have been used to define a type; this is necessary to be able to generate a virgin type tag for the next type declaration.

An evironment maps each variable identifier into a relative L-value, each function and procedure identifier into a procedure value and each label (numeral) into a continuation. In addition for each identifier and label the lexical level of its definition is given. Finally, a component $L_r \to T$ keeps track of used relative locations.

$$\begin{aligned} \rho \in U = &(Id \to Lv_r) \times \\ &(Id \to P) \times \\ &(Id \to B) \times \\ &(Num \to G) \times \\ &(Num \to B) \times \\ &(L_r \to T) \times \\ &B \end{aligned}$$

5.1.2. Types and modes

The domain Ty of types provides an "internal representation" for user defined types. We use a definition similar to that of the abstract syntax. A $\tau \in Ty$ is enclosed in $\lceil \ldots \rceil$, similar to elements of the abstract syntax which are enclosed in $[\![\ldots]\!]$.

Each $\tau \in Ty$ has various fields, one of these is a type tag that uniquely identifies the type. An exception is the *nil* type which is a special pointer type. It is special because *nil* is common to all pointer types and has to be treated differently during type checking. In the following definition products are not strict! The domain Ty contains infinite objects (e.g. recursive types).

$$\begin{aligned}\tau = &\lceil \nu : sub : \iota_1 : \iota_2 \rceil \mid \lceil \nu :\uparrow \tau \rceil \mid \lceil nil \rceil \\ &\mid \lceil \nu : array : \tau_1 : \tau_2 \rceil \mid \lceil \nu : record : \langle I_i : \tau_1, \ldots, I_n : \tau_n \rangle \rceil\end{aligned}$$

Each identifier and expression in *LS* is assigned a mode. For example, a mode specifies, whether an object is a constant, a variable, a value, a procedure and so on.

A mode $\mu \in Md$ is enclosed in $\lfloor \ldots \rfloor$; we have the following modes:

- $\lfloor var : \tau \rfloor$ a variable of type τ.
- $\lfloor vap : \tau \rfloor$ a var parameter inside a procedure. Its value is a variable of type τ.
- $\lfloor val : \tau \rfloor$ a value of type τ.
- $\lfloor type : \tau \rfloor$ a type identifier denoting type τ.
- $\lfloor const : \iota : \tau \rfloor$ a constant of type τ with value ι.
- $\lfloor proc : \mu_1, \ldots, \mu_n \rfloor$ a user defined procedure with parameter modes $\mu_1, \ldots, \mu_n$.
- $\lfloor sproc \rfloor$ a special procedure. A special procedure is a predefined procedure. These are treated differently from user defined procedures to allow for more flexible parameter checking (e.g. predefined procedures can be generic, *new* is an example).
- $\lfloor afun : \mu_1, \ldots, \mu_n : \tau \rfloor$ an active function with parameter modes $\mu_1, \ldots, \mu_n$ and result type τ. A function is active inside its own body; it is passive otherwise. The distinction is relevant since it is possible to assign to an active function but not to a passive function.
- $\lfloor pfun : \mu_1, \ldots, \mu_n : \tau \rfloor$ a passive function.
- $\lfloor sfun \rfloor$ a special function (see remark under special procedure).

5.1.3. Auxiliary functions, static semantics

The following is a list of functions used in defining the statics semantics of *LS*

with a brief description.

distinct $\in D^* \to T$ is defined for any flat domain D for which an equality test is defined. It is used to test lists of numbers and identifiers for distinctness.

typetag $\in Ty \to Tg$ selects the type tag of a type.

A type is an index type if it is a subrange (note, that integers are a subrange $int_{-\infty}..int_{+\infty}$). *isindex* $\in Ty \to T$ returns TT for index types.

A type is returnable if it can be the result type of a function, i.e. *isreturnable* $\in Ty \to T$. In *LS* we allow index types, pointers, and the nil type to be returned by functions.

overlaps, *contains* $\in Ty \to Ty \to T$, for two subranges these functions determine whether they overlap or whether one is contained in the other. They are useful in the definition of assignments: two subranges that do not overlap cannot be assigned; if the right hand side is contained in the left hand side, no runtime check is required.

union $\in Ty \to Ty \to Ty$ constructs the smallest subrange that contains two subrange types.

type $\in Md \to Ty$ selects the type of a mode. For example *type* $\lfloor var : \tau \rfloor = \tau$. *type* is undefined for procedure modes and special functions.

equal $\in Ty \to Ty \to T$ tests two types for equality.

compatible $\in Ty \to Ty \to T$, two types are compatible if there is a possible runtime check that allows these types to be assigned to each other. For example, equal types are compatible; two subranges that overlap are compatible; a pointer type is compatible with the nil type.

assignable $\in Md \to Md \to T$; mode μ_2 is assignable to mode μ_1 if μ_1 is a variable, if types of μ_1 and μ_2 are compatible, and if the type of μ_2 is returnable. *LS* only allows assignment of objects that are also returnable as function results. Arrays or records cannot be assigned.

If an expression has mode μ_2, *passable* $\in Md \to Md \to T$ determines if it can be passed to a formal parameter that has mode μ_1. For var parameters *LS* requires type equality; in particular, subranges have to match exactly. For value parameters the two modes have to be assignable; i.e. complex data objects cannot be passed as value parameters.

isvar, *isval*, $\in Md \to T$ check whether a mode is a value or a variable.

isbool, *isint* $\in Ty \to T$ test if a type is integer or boolean.

sp $\in Id \to Md^* \to T$ checks a call to a special procedure for validity. Given the name of the special procedure and the modes of the actual parameters *sp* returns TT if this call is valid. Note, that this is a very flexible schema; for example a procedure *print* could be made generic for all types; also print could take a variable number of parameters for each call. We assume, however, that *print* has only one

integer argument; extensions are easy to add to our compiler.

$sf \in Id \to Md^* \to Md$ is similar to *sp* but checks special functions instead. The result of sf is the mode of the object returned by the special function.

truemode, $falsemode \in Md$ are modes of the constants *true* and *false*.

$intconst \in I \to Md$ contructs an integer constant of a given value.

integer $\in Ty$ is the type integer.

boolean $\in Ty$ is the type boolean.

$mkevalmode \in Md \to Md$ makes a value mode with the same type as a given mode.

$w \in \Omega \to Md \to Md \to Md$, $o \in O \to Md \to Md$ are similar to sf; they checks binary and unary operators for validity.

5.2. Static semantics

5.2.1. Declarations

A list of declarations is checked by d for its correctness. d also updates the environment to reflect the new declarations. We have $d \in Dec \to U_s \to U_s$. To treat constant, type, and var declarations d uses the functions $dc \in Cdef \to U_s \to U_s$, $dt \in Tdef \to U_s \to U_s \to U_s$, and $dv \in Vdec \to U_s \to U_s$.

One of the more interesting points in d is the treatment of recursive types. In d we have the clause

$$d[\![type\Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma = fix(dt^*[\![\Delta_{t1}; \ldots; \Delta_{tn})]\!]\varsigma).$$

Note, that dt requires two environments as argument. To get a feel for the meaning of the fixed point, consider the limit case, where we have $d[\![type\Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma = x$ such that $x = dt^*[\![\Delta_{t1}; \ldots; \Delta_{tn})]\!]\varsigma\ x$. If any of the Δ_{ti} contains a pointer type, the reference will be resolved by looking into x instead of ς, whereas ς is used for all other identifiers: they have to be defined in a global scope.

Another interesting case is that of procedure definitions.

$$\begin{aligned} d[\![procedure\ I(\Pi_1, \ldots, \Pi_n); \Theta]\!]\varsigma_0 = \ & \textbf{let } \langle(\mu_1, \ldots, \mu_n), \varsigma_n\rangle :: p^*[\![\Pi_1, \ldots, \Pi_n]\!]\varsigma_0 \textbf{ in} \\ & \textbf{if } distinct(i^*[\![\Pi_1, \ldots, \Pi_n]\!] \cdot i[\![\Theta]\!]) \textbf{ then} \\ & \textbf{if } g[\![\Theta]\!]\varsigma_n[\lfloor proc(\mu_1, \ldots, \mu_n)\rfloor/I] \textbf{ then} \\ & \quad \varsigma_0[\lfloor proc(\mu_1, \ldots, \mu_n)\rfloor/I] \end{aligned}$$

p^* is used to determine the modes of the parameters as well as the environment ς_n resulting from ς_0 after entering the parameters with their modes. If all identifiers used as parameters and in the procedure body are distinct and if the procedure body

is semantically correct (see definition of g) in ς_n, then the resulting environment is obtained by entering the corresponding procedure mode for I in ς_0.

The remaining cases are straightforward.

5.2.2. Types

$t \in Typ \to U_s \to U_s \to (Ty \times U_s)$ determines the meaning of a type in the program. A given type denotation is evaluated and a representation ($\in Ty$) is constructed. Two points are to be observed here. t changes the environment in two ways. First, an enumeration defines a set of identifiers as constants of this enumeration types; these have to be entered by t in the environment. Furthermore, to represent new types, t uses up type tags which in turn have to be marked as being in use in the new environment.

The second point to observe is that like dt, t too takes two environments as argument. The reason is exactly the same here: t uses the first environment in all cases except to determine the type pointed to by a pointer type declaration.

5.2.3. Labels and identifiers

$j \in Com \to Num^*$ returns the list of all labels defined on the top level of a given command.

k returns the list of all procedures and functions defined in a block.

Function i returns all identifiers defined in a given language construct. It is defined for several syntactic domains: $i \in (Par + Stm + Dec + Typ) \to Id^*$.

5.2.4. Expressions

The function $e \in Exp \to U_s \to Md$ evaluates an expression in a given static environment and returns the mode of the expression. In computing this mode e checks if all declaration and type constraints are satisfies. If this is not the case e will return $\perp$. We will explain some typical clauses of the definition.

For an integer constant we have $e[\![N]\!]\varsigma = \lfloor const : n : \lceil int : sub : n : n \rceil \rfloor$, where n is the value of the constant N (given as the token value). That is, the mode of a constant is the *constantmode* with the value being the value of the constant and the type being the smallest possible subrange containing this constant.

The mode of a binary operator is given by the function w applied to the modes of the operands: $e[\![E_0\ \Omega\ E_1]\!]\varsigma = w[\![\Omega]\!]\langle e[\![E_0]\!]\varsigma, e[\![E_1]\!]\varsigma\rangle$

For an indexing operation to be well typed e has to check that

- the indexed expression is of an array type,
- the type of the indexing expression is compatible with the index type of the array type.

The resulting mode is a var in any case: there are no values or constants of array type. The corresponding definitional clause is:

$$\begin{aligned} e[\![E_0[E_1]]\!]\varsigma = \textbf{if } \lceil \nu : \mathit{array}\ \tau_1\ \mathit{of}\ \tau_2 \rceil :: \mathit{type}\ e[\![E_0]\!]\varsigma \textbf{ then} \\ \textbf{if } \mathit{compatible}\ \tau_1(\mathit{type}\ e[\![E_1]\!]\varsigma) \textbf{ then} \\ \textbf{if } \mathit{isvar}\ e[\![E_0]\!]\varsigma \textbf{ then } \lfloor \mathit{var} : \tau_2 \rfloor \textbf{ else} \\ \textbf{if } \mathit{isvap}\ e[\![E_0]\!]\varsigma \textbf{ then } \lfloor \mathit{var} : \tau_2 \rfloor . \end{aligned}$$

Other expressions are defined in a straightforward way following the above schema.

5.2.5. Statements

Statements are checked by $g \in \mathit{Stm} \to U_s \to T$ for their semantic validity. The definition is straightforward, for example assignments are defined as

$$g[\![E_1 := E_2]\!]\varsigma = \mathit{assignable}(e[\![E_1]\!]\varsigma)(e[\![E_2]\!]\varsigma).$$

That is, an assignment is legal if the modes of left and right hand side are assignable.

A slightly more complex example are while loops. There are two semantic restrictions: the test must yield a boolean result and the body of the while statement must be valid. For the latter test c is used to evaluate a command. Note, that c is passed a modified environment in which all labels in the while body are defined; this effectively make the body of the while loop a new scope for the purpose of labels. It becomes impossible to jump into a while body.

$$\begin{aligned} g[\![\mathit{while}\ E\ \mathit{do}\ \Gamma\ \mathit{od}]\!]\varsigma = & \mathit{isbool}(\mathit{type}\ e[\![E]\!]\varsigma) \wedge \\ & \mathit{distinct}(j[\![\Gamma]\!]) \wedge \\ & c[\![\Gamma]\!]\varsigma[\langle \ldots, \mathit{true}, \ldots \rangle / j[\![\Gamma]\!]]. \end{aligned}$$

5.2.6. Commands

Commands are lists of labeled or unlabeled statements. A list of statements is semantically valid, if all individual statements are valid. Thus for $c \in \mathit{Com} \to U_s \to T$ we have the simple definition:

$$\begin{aligned} c[\![N : \Theta]\!]\varsigma &= g[\![\Theta]\!]\varsigma \\ c[\![\Theta_1; \Theta_2]\!]\varsigma &= c[\![\Theta_1]\!]\varsigma \wedge c[\![\Theta_2]\!]\varsigma \\ c[\![\Theta]\!]\varsigma &= g[\![\Theta]\!]\varsigma. \end{aligned}$$

5.3. Dynamic semantics

Meaning functions of the dynamic semantics are denoted by capital script letters $\mathcal{E}$, $\mathcal{D}$ and so on.

5.3.1. Auxiliary functions

By introducing generalized continuations operations on continuations C have to be extended to operate on G_d. Suppose we are given $\gamma \in G_d = V \to \ldots \to V \to S \to A$. We want to define a continuation $\acute{\gamma}$ which reflects a change $f \in S \to S$ to the store. I.e. we want

$$\gamma\epsilon_1 \ldots \epsilon_n(f\ \sigma) = \acute{\gamma}\epsilon_1 \ldots \epsilon_n\sigma.$$

The function $change \in G_d \to (S \to S) \to G_d$ computes $\acute{\gamma}$ given f and γ.

$update \in G \to G$ describes the effect on a continuation of assigning a value to a location:

$$update\ \gamma = \lambda\chi\epsilon\alpha.change(\gamma\chi)(\lambda\sigma.\sigma[\epsilon/\alpha]).$$

$content \in G \to G$ decscribes the effect of accessing a location in memory. Let γ be a continuation that takes n arguments $\in V$, then *content* is defined such that

$$content\ \chi\gamma = \lambda\chi\alpha\epsilon_2 \ldots \epsilon_n\sigma.(\gamma\chi)(\sigma\alpha)\epsilon_2 \ldots \epsilon_n\sigma.$$

$cond \in G \to G \to G$ defines a "conditional continuation". Given arguments γ_1 and γ_2 *cond* defines a continuation which takes an argument ϵ and depending on the value of ϵ applies either $\gamma 1$ or γ_2. We define ϵ to be true if $\epsilon \mid I$ is zero.

$$cond\gamma_1\gamma_2 = \lambda\chi\epsilon.\ \textbf{if}\ (\epsilon \mid I) = 0\ \textbf{then}\ \gamma_1\chi\ \textbf{else}\ \gamma_2\chi.$$

$binop \in Op \to G \to G$ and $unop \in Mop \to G \to G$ define the meaning of binary and unary operators respectively.

$Sp \in Id \to Md^* \to G \to G$ defines the meaning of special procedures. In our case we have *print* and *new*. Similarly, $Sf \in Id \to G \to G$ defines the meaning of special functions; in *LS* we have *eof* and *read*.

$verifys \in I \to I \to G \to G$ Given two index values and a continuation *verifys* verifies that a value applied to the continuation is in the subrange defined by the two index values. If this condition is not satisfied *verifys* will cause a runtime error. Similarly, $verifyn \in G \to G$ will check if a value supplied to a continuation is a pointer not equal to *nil*. *verifyn* is used to ensure that no nil pointer is dereferenced.

$index \in G \to G$ and $select \in Id \to G \to G$ describe the effect of indexing an array and selecting a component of a record.

Generalized continuation cause one slight problem in the definition of jumps. Suppose we describe the meaning of *goto n* where the continuation following this jump is γ and the continuation of the label n is $\acute{\gamma}$. In certain cases (e.g. if the jump leaves a function) the domains of γ and $\acute{\gamma}$ may differ and require different numbers of arguments $\in V$. Thus, before jumping we have to *adjust* the continuation we

jump to. As mentioned above, we may look the V arguments of γ as a stack. It is well known to the compiler constructor that jumps out of functions require adjustment of the runtime stack. This fact is reflected in the formal definition. $adjust \in G \to N \to G$ adjusts a given continuation to take n more arguments.

$args \in G \to N$ simply determines the number of V arguments of a continuation.

5.3.2. Memory allocation

We define functions to (i) allocate static memory, (ii) dynamic memory on the heap, and (iii) functions to initialize newly allocated memory. In each case we distinguish objects that occupy one location and complex data objects stored in several locations.

new, *newl*, and *newa* allocate static locations. *new* allocates a single location: $new \in U \to (Lv_r \times U)$. $newl \in Ty \to U \to (Lv_r \times U)$ takes a type as additional argument; it allocates memory for any type. Finally $newa \in I \to I \to Ty \to U \to (Lv_r \times U)$ allocates memory for arrays, given the index values which define the index range and the element type of the array.

The second set of functions is *clear* and *cleara*. These do not allocate memory; rather they initialize newly allocated memory to 0. In *LS* all variables are initialized to 0. Alternatives would be to leave their values arbitrary and have the program behave nondeterministically or to initialize variables to *undefined* and causing a runtime error whenever an uninitialized variable is accessed. The former requires a more complicated definition using power domains [Pl76,Sm78]. The latter was avoided since the resulting semantics cannot efficiently be implemented on most machines; it requires an additional bit in all memory words to indicate initialization.

The third set of functions allocates dynamic locations (on the heap). We have $heap \in S \to (Lv_d \times S)$, $heapa \in I \to I \to Ty \to S \to (Lv_d \times S)$, and $heapl \in Ty \to S \to (Lv_d \times S)$ which are analogous to *new*, *newa*, and *newl* but in addition initialize the alloacated memory.

5.3.3. Declarations

Declarations cause a change in the environment: declared variables are allocated, procedure and function values are entered in the environment. In addition declarations require the execution of initialization code.

Allocation of memory for variables is defined by the functions

$$\begin{aligned} \mathcal{V} &\in Decl \to U_s \to U \to U \\ \mathcal{V}^* &\in Decl^* \to U_s \to U \to U \\ \mathcal{V}_v &\in Vdec \to U_s \to U \to U. \end{aligned}$$

Initialization of variable is described by

$$\begin{aligned}
\mathcal{D} &\in Decl \to U_s \to U \to G \to G \\
\mathcal{D}^* &\in Decl^* \to U_s \to U \to G \to G \\
\mathcal{D}_v &\in V\,dec \to U_s \to U \to G \to G \\
\mathcal{D}_v^* &\in V\,dec^* \to U_s \to U \to G \to G.
\end{aligned}$$

Inside a procedure or function parameters are treated very much like declarations: we allocate memory for each parameter; we initialize each parameter. The difference to declarations is that one location suffices for each parameter; furthermore, parameters are initialized to the actual parameters rather than 0. Allocation is done by $\mathcal{Q} \in Par \to U_s \to U \to U$ and $\mathcal{Q}^* \in Par^* \to U_s \to U \to U$. Parameters are initialized by $\mathcal{P} \in Par \to U_s \to U \to G \to G$ and $\mathcal{P}^* \in Par^* \to U_s \to U \to E^* \to G \to G$.

$\mathcal{F} \in Decl \to U_s \to U \to G^*$ and $\mathcal{F}^* \in Decl^* \to U_s \to U \to G^*$ return the list of procedure and function values defined by a list of declarations. The way these function and procedure values are entered in the environment is described under statements where we discuss the meaning of blocks.

Let us explain the meaning of procedure; functions are treated similarly. We have the definition:

$$\begin{aligned}
\mathcal{F}[\![\mathit{procedure}\ I(\Pi_1,\ldots,\Pi_n);\Theta]\!]\varsigma\rho &= \langle\pi\rangle \\
&\textbf{where } \pi = \lambda\gamma.\mathit{enter}(n+1);\mathcal{P}^*[\![\Pi_1,\ldots,\Pi_n]\!]\varsigma_1\rho_1; \\
&\qquad \mathcal{B}[\![\Theta]\!]\varsigma_2\rho_1;\mathit{exit}\ \gamma \\
&\textbf{where } \varsigma_2 = \varsigma_1[\lfloor \mathit{proc} : \mu_1,\ldots,\mu_n \rfloor / I] \\
\textbf{where } \langle\langle\mu_1,\ldots,\mu_n\rangle,\varsigma_1\rangle &= p^*[\![\Pi_1,\ldots,\Pi_n]\!]\varsigma \\
&\textbf{where } \rho_1 = \mathcal{Q}^*[\![\Pi_1,\ldots,\Pi_n]\!]\varsigma_1(\mathit{next}\ \rho) \\
&\textbf{where } n = \mathit{level}\ \rho.
\end{aligned}$$

ς_2 is the static environment in which formal parameters are bound to their mode and the procedure identifier is bound to the correct procedure mode. The environment ρ_1 has memory allocated for parameters.

The value of the procedure then is given by creating an new frame (*enter*), initializing all parameters, executing the body of the procedure in ς_2 and ρ_2, and discarding the frame for this procedure incarnation (*exit*).

5.3.4. Expressions

Expressions are evaluated by $\mathcal{E}$, $\mathcal{A}$, $\mathcal{L}$, and $\mathcal{R}$. Here, $\mathcal{E}$ evaluates an expression without any coercion. $\mathcal{A}$ takes a mode as additional parameter and coerces the expression to this particular mode. $\mathcal{L}$ and $\mathcal{R}$ essentially are special cases of $\mathcal{A}$ and are used to evaluate expressions on the left hand side and right hand side of an assignment respectively. All definitions are straightforward.

$\mathcal{E}$ does constant evaluation, i.e. if an expression is of constant mode its value is this constant. Although not strictly necessary, we made constant evaluation part of the definition. If it were not, constant evaluation could be an optimization. However, care has to be taken so that for instance the evaluation of $1/0$ causes an error at the point defined by the formal semantics. Our definition of *LS* states that this zero division is to be detected at compile time.

$\mathcal{E}$ and $\mathcal{A}$ check for runtime errors such as invalid index and dereferencing of nil–pointers. This is signified by the clauses:

$$\hat{\mathcal{E}}[\![E\uparrow]\!]\varsigma\rho\gamma = \mathcal{R}[\![E]\!]\varsigma\rho; verifyn\ \gamma$$

and

$$\begin{aligned}\mathcal{A}[\![E]\!]\varsigma\mu\rho\gamma = \ & \textbf{if } \mu :: \lfloor var : \tau \rfloor \textbf{ then } \mathcal{L}[\![E]\!]\varsigma\rho\gamma \textbf{ else} \\ & \textbf{if } \mu :: \lfloor val : \tau \rfloor \textbf{ then} \\ & \quad (\textbf{ if } \acute{\tau} :: \lceil \nu : sub : \epsilon_0 : \epsilon_1 \rceil \textbf{ then} \\ & \qquad \textbf{if } contains\ \tau\acute{\tau} \textbf{ then } \mathcal{R}[\![E]\!]\varsigma\rho\gamma \\ & \qquad\quad \textbf{else } \mathcal{R}[\![E]\!]\varsigma\rho; verifys\ \epsilon_0\epsilon_2\gamma \\ & \qquad \textbf{else } \mathcal{R}[\![E]\!]\varsigma\rho\gamma) \\ & \quad \textbf{where } \acute{\tau} = type(e[\![E]\!]\varsigma).\end{aligned}$$

5.3.5. Statements

In *LS* we distinguish commands and statements. The reason is to be able to handle jumps properly. In particular we have to rule out that jumps lead into statements.

The meaning of commands is trivially defined as

$$\boxed{\mathcal{C} \in Com \to U_s \to U \to G \to G}$$

$$\begin{aligned}\mathcal{C}[\![N : \Theta]\!]\varsigma\rho\gamma &= \mathcal{B}[\![\Theta]\!]\varsigma\rho\gamma \\ \mathcal{C}[\![\Theta]\!]\varsigma\rho\gamma &= \mathcal{B}[\![\Theta]\!]\varsigma\rho\gamma \\ \mathcal{C}[\![\Gamma_0;\Gamma_1]\!]\varsigma\rho\gamma &= \mathcal{C}[\![\Gamma_0]\!]\varsigma\rho; \mathcal{C}[\![\Gamma_1]\!]\varsigma\rho\gamma\end{aligned}$$

A special function $\mathcal{C}l$ defines the meaning of a command that appears as a component of a composite statement such as the then part of an if-statement. Such a command has to be evaluated in an environment where labels defined in this command are bound to proper continuations. These labels are not bound in an enclosing scope to prohibit jumps into composite statements:

$$\boxed{\mathcal{C}\ell \in Com \to U_s \to U \to C \to C}$$

$$\begin{aligned}\mathcal{C}\ell[\![\Gamma]\!]\varsigma\rho\gamma &= \mathcal{C}[\![\Gamma]\!]\hat{\varsigma}\hat{\rho}\gamma \\ &\quad \textbf{where } \hat{\varsigma} = \varsigma[\langle \ldots, true, \ldots\rangle/j[\![\Gamma]\!]], \\ &\quad \hat{\rho} = fix(\lambda\hat{\rho}.\rho[\mathcal{J}[\![\Gamma]\!]\hat{\varsigma}\hat{\rho}\gamma/j[\![\Gamma]\!]]).\end{aligned}$$

$\mathcal{J} \in Com \to U_s \to U \to G \to G^*$ returns a list of continuations corresponding to the labels defined in a command.

$$\begin{aligned}\mathcal{J}[\![N : \Theta]\!]\varsigma\rho\gamma &= \langle \mathcal{B}[\![\Theta]\!]\varsigma\rho\gamma\rangle \\ \mathcal{J}[\![\Gamma_1;\Gamma_2]\!]\varsigma\rho\gamma &= \mathcal{J}[\![\Gamma_1]\!]\varsigma\rho(\mathcal{C}[\![\Gamma_2]\!]\varsigma\rho\gamma)\cdot\mathcal{J}[\![\Gamma_2]\!]\varsigma\rho\gamma \\ \mathcal{J}[\![\Theta]\!]\varsigma\rho\gamma &= \langle\rangle.\end{aligned}$$

Semantics of statements ($\mathcal{B}$) is conventional, however, we should mention two points here. For jumps we have to adjust the continuation as mentioned before. This is done by

$$\mathcal{B}[\![goto\ N]\!]\varsigma\rho\gamma = adjust(\rho[\![N]\!])(args\ \gamma - args(\rho[\![N]\!])).$$

The second point of interest are declarations. The meaning of a block is to

- initialize locally declared variables followed by
- execution of the body in the correct environment.

The static environment is determined as for static semantics. In the environment we have to allocate new storage for local variables and bind procedure and function values and label continuations. The latter step requires to construct a fixed point: the continuation at a label depends on continuations of other labels. Similarly, procedure and function values depend on other procedure and function values (for recursive calls) as well as label continuations. Our definition handles mutually recursive procedures.[4]

$$\begin{aligned}\mathcal{B}[\![\Delta^*; begin\ \Gamma\ end]\!]\varsigma\rho\gamma &= \mathcal{D}^*[\![\Delta^*]\!]\hat{\varsigma}\hat{\rho}; \mathcal{C}[\![\Gamma]\!]\hat{\varsigma}\hat{\rho}\gamma \\ &\quad \textbf{where } \hat{\varsigma} = d[\![\Delta^*]\!]\varsigma[\langle \ldots, true, \ldots\rangle/j[\![\Gamma]\!]], \\ &\qquad \hat{\rho} = fix\Big(\lambda\hat{\rho}.\ddot{\rho}[\mathcal{J}[\![\Gamma]\!]\hat{\varsigma}\hat{\rho}\gamma/j[\![\Gamma]\!]][\mathcal{F}^*[\![\Delta^*]\!]\hat{\varsigma}\hat{\rho}/k[\![\Delta^*]\!]]\Big) \\ &\quad \textbf{where } \ddot{\rho} = \mathcal{V}^*[\![\Delta^*]\!]\hat{\varsigma}\rho.\end{aligned}$$

6. The target language *LT*

In this section we describe the target language *LT*. First we present the ar-

4.) these are disallowed by the static semantics

chitecture of a hypothetical machine and give an informal description of the language. Strictly speaking it is not necessary to define the machine architecture; the language can be defined independent of a particular implementation. Having a concrete machine to talk about will make out explanations easier to understand and transparent.

Following the informal discussion we present a formal denotational definition of *LT*. Only some interesting parts are discussed here; the complete semantics is presented in appendix 2.

6.1. A hypothetical machine

6.1.1. Design decisions

Ease of translation was one important concern. Therefore, the language contains fairly "high level" concepts. However, it will be simple to verify correct translation of *LT* in a lower machine language, for instance for a register machine.

The underlying hypothetical machine is a stack machine. The key idea underlying stack architectures is to implement some components of the runtime system for Algol–like languages in hardware [Or73]. The standard runtime organization as well as the machine architecture of the B6700 (a stack machine) [BC71, BC72, BC73] combine conceptually different objects into one stack. This stack is used to (*i*) evaluate expression, (*ii*) allocate static memory, and (*iii*) store administrative information about return addresses of procedures, static and dynamic links of procedure invocations.

In our machine these different objects of the stack are clearly separated which simplifies our proofs. But at the same time it will not be very difficult to encode separate components of our machine in one stack if efficieny dictates so.

6.1.2. Architecture

Our hypothetical machine (see figure 2) consists of the following components.

- An infinite memory capable of storing infinite precision integers.
- A stack used for expression evaluation (potentially infinite)
- a display mechanism (see below).
- one input and one output file.

Our machine knows two kinds of addresses: relative and absolute addresses, both integers. Accessing memory can only be done with absolute addresses. Relative addresses can be converted into absolute ones using information contained in the display (see below).

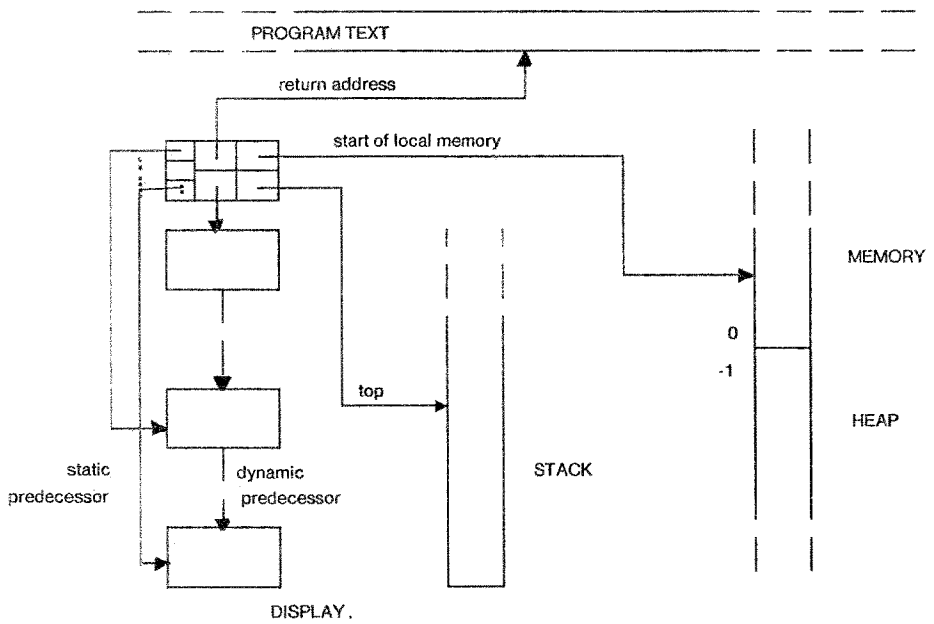


Fig. 2

The memory is divided into two parts, negative and positive addresses. The part with negative addresses will be used as heap to store dynamically allocated objects. Memory with positive addresses is administered in a stack-like fashion; i.e. memory allocated in this part becomes available when the scope in which it was allocated is left.

The display mechanism of our hypothetical machine models the domain X of the source language definition. But the domain D of target display contains more information than X. For each display we have:

- The lexical level of the currently executed procedure.
- A pointer to the displays of all static predecessors.
- The base address of each static predecessor (not shown in the picture of our machine).
- A pointer to the display of the dynamic predecessor.
- The length of the stack upon entry of the current procedure. This information is used by the *JUMP* instruction if the jump leads out of a procedure or function.

- The return address of the current procedure.

Our design of the display mechanism is not very efficient, but this is not the point of our discussion. More efficient architectures are possible that realize the same semantics.

6.1.3. Instructions

Let us briefly give an informal description of the instructions available on our hypothetical machine. Instructions have varying format; they may have none or several parameters. Generally, all instructions operate on the stack. That is, arguments are obtained from the stack and results are put back on the stack.

- *LIT* n pushs the constant n on the stack.
- *LOAD* uses the top of the stack as an address and push the content of the corresponding location.
- *ADDR* n m takes the relative address n and the lexical level m and compute and absolute address from the current display. The result is pushed on the stack.
- *STORE* pops a value and an absolute address from the stack and updates the address with the value.
- *CALL* l n m k calls the procedure starting at label l defined on lexical level n. m is the number of parameters of this procedure and k is the number of static memory locations allocated in the calling environment. *CALL* creates a new display, stores the return address, computes the starting location of the available memory, and marks the level of the stack (current stack length minus the number of parameters).
- *EXIT* exits a procedure or function. This requires to make the dynamic predecessor of the display the new display and branching to the return address of the procedure. The stack remains unchanged, this allows a function result to be pushed on the stack before executing *EXIT*. This result is then available to the callee.
- *NCOND* l inspects the top element of the stack. If it is false ($\neq 0$) then it branchs to label l. This instruction assumes that the label is defined on the current lexical level.
- *HOP* l unconditionally branches to label l; l is assumed to be on the current lexical level.
- *JUMP* l n unconditionally branches to label l defined on lexical level n. This requires to make the display of level n the current display and to adjust the

stack to a length indicated in this display.

- *LABSET* n is a meta instruction and is used to define a label. Its semantics is a no-op.
- *STOP* stops program execution. This is the normal termination; "running" off the end of a list of instructions will cause an error.
- *UNOP* n performs the unary operation number n using the top of the stack as argument and pushing the result. We deliberately leave the precise nature of the unary and binary operations undefined. However, they have the same meaning as those in the source language. E.g. + in the source language has the same semantics as addition on the target machine.
- *BINOP* n performs binary operation n on the two top elements.

For the sake of simplicity we assume that the machine has standard input output instructions. In practice these instructions would be part of a runtime system.

- *EOF* pushes *true* (0) on the stack if the input file is empty, *false* (1) otherwise.
- *OUTP* prints the top of the stack on the output file.
- *INPT* pushes the first element of the input file on the stack.

6.2. Formal definition of *LT*

As mentioned earlier, no concrete syntax is required since we are not interested in parsing programs written in the target language. Also, we are not interested in checking the static semantics of *LT* programs. We merely define the dynamic semantics. If a program is not well formed, for example if a label is multiply defined, the meaning of this program will be $\perp$.

6.2.1. Abstract syntax

The definition of the abstract syntax is straight forward. We have the domains of labels, numeral, instructions, and programs.

6.2.2. Semantic domains

Integers are as usual; they model values stored in memory as well as addresses. Stacks S are finite lists of integers, $S = V^*$.

Environments map labels into continuations much like in *LS*. Since there are no variables this is the only information in environments.

Displays are defined as

$$\delta \in D = N \to D \times N \to N \times D \times N \times G \times N.$$

The components have the following meaning (in this order)

- Mapping from lexical levels into displays of the static predecessors.
- Mapping from lexical levels to starting addresses of the corresponding memory frame.
- The dynamic predecessor.
- The current lexical level.
- Continuation to be used upon procedure exit.
- Stack length of on entry to the current procedure.

The memory $M = (N \to N) \times N^* \times N^* \times N$ is a mapping from addresses to values $(N \to N)$, the input and output file, and the first available location of the heap.

Continuation in LT are mappings from the current machine state into answers. A machine state is characterized by the current display, the stack, and the memory. We curry and write $G = D \to S \to M \to A$.

The domain of answers is identical to answers of LS; an answer is either an output file or a runtime error. $A = N^* + \{eoferror, nostop, \ldots\}$

6.2.3. Semantic equations

The valuations for individual instructions are straightforward. We use functions *up* and *down* do create a new display (enter a procedure) and adjust the stack after global jumps.

Labels are handled similar as in LS. The function $\mathcal{L}$ returns a list of continuation holding at labels defined in the program. These continuation are bound to the labels in the environment. As in LS we have to construct a fixed point since the continuation at a label will depend on the continuation already bound to other labels in the environment.